The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 1: Introduction to PostgreSQL

1.1 What is PostgreSQL?

PostgreSQL is an open-source Relational Database Management System (RDBMS). It stores data in tables and understands SQL. It is widely used in production systems because it is reliable, feature-rich and standards compliant.

1.2 Why PostgreSQL?

Reasons developers choose PostgreSQL:
• Open source
• ACID compliant
• Excellent SQL support
• Strong indexing and query optimizer
• Supports JSON in addition to relational data
• Used by startups and large companies alike

1.3 SQL vs PostgreSQL

SQL is the language you write.
PostgreSQL is the software that executes SQL and manages the database.

Application
↓
PostgreSQL
↓
Database Files

1.4 PostgreSQL vs MySQL

PostgreSQL focuses on standards, advanced SQL features and extensibility.
MySQL is also popular and widely used, especially for web applications.
Both are excellent relational databases.

1.5 PostgreSQL Components

Database → Schemas → Tables → Rows → Columns

Example:
Database: practice
Table: users
Columns: id, name, email
Rows: actual user records

1.6 Development Workflow

Typical backend workflow:
1. Create database
2. Create tables
3. Insert data
4. Query data
5. Connect Node.js using pg
6. Build APIs
7. Later introduce Prisma ORM

Interview Questions

1. What is PostgreSQL?
2. Is PostgreSQL a DBMS or an RDBMS?
3. Difference between SQL and PostgreSQL?
4. Why would you choose PostgreSQL?
5. Name some companies that use PostgreSQL.

Practice

1. Install PostgreSQL.
2. Create a database named practice.
3. Open pgAdmin and identify the databases.
4. Explain the difference between a database and a table.

Chapter Summary

You learned:
• PostgreSQL basics
• Why PostgreSQL is popular
• SQL vs PostgreSQL
• PostgreSQL architecture
• Typical backend workflow

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 2: Installing PostgreSQL, pgAdmin & psql

Installing PostgreSQL

Download PostgreSQL from the official website. During installation choose a superuser password, keep the default port (5432 unless you have a reason to change it), and install pgAdmin and the command-line tools.

Installation Details

Remember your username (often postgres), password, host (localhost), and port (5432). These values are later used in Node.js when creating a database connection.

What is pgAdmin?

pgAdmin is a graphical interface for PostgreSQL. You can create databases, tables, run SQL queries, inspect data, manage users and indexes without using the terminal.

What is psql?

psql is PostgreSQL's command-line interface. Common commands:
\l - list databases
\c database_name - connect to database
\dt - list tables
\q - quit.

pgAdmin vs psql

pgAdmin is easier for beginners and visual inspection. psql is faster for experienced users and useful on servers without a graphical interface.

Verification

Verify that PostgreSQL is running, pgAdmin opens, psql connects successfully, and you can create a test database.

Common Problems

Wrong password, PostgreSQL service stopped, PATH not configured for psql, another process using port 5432, or firewall/network issues.

Interview Questions

1. What is pgAdmin?
2. What is psql?
3. What is PostgreSQL's default port?
4. What connection details are required by Node.js?
5. When would you prefer psql over pgAdmin?

Practice

1. Install PostgreSQL.
2. Open pgAdmin.
3. Create a database named practice.
4. Connect using psql.
5. Run \l and \dt.

Chapter Summary

You learned PostgreSQL installation, pgAdmin, psql, important connection settings, verification steps and common installation issues.

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 3: Creating Databases, Tables & Data Types

3.1 Creating a Database

A database is a container that holds tables and other database objects.

SQL:
CREATE DATABASE practice;

Connect to it:
\c practice

3.2 Creating Tables

Tables store related information.

Example:

CREATE TABLE users(
id SERIAL PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(255)
);

3.3 Common PostgreSQL Data Types

INTEGER / INT - Whole numbers
SERIAL - Auto increment integer
VARCHAR(n) - Variable length text
TEXT - Unlimited text
BOOLEAN - true/false
DATE - Calendar date
TIMESTAMP - Date and time
NUMERIC / DECIMAL - Precise decimal values

3.4 Choosing the Right Data Type

Use INT for ids and counts.
Use VARCHAR for names and emails.
Use TEXT for descriptions.
Use BOOLEAN for status values.
Use DATE or TIMESTAMP for dates.
Choose the smallest suitable type that satisfies the requirement.

3.5 Inserting Sample Data

INSERT INTO users(name,email)
VALUES
('Nikhil','nikhil@gmail.com'),
('Rahul','rahul@gmail.com');

3.6 Viewing Data

SELECT * FROM users;

```
SELECT name,email
FROM users;
```

3.7 Modifying Tables

Add a column:

```
ALTER TABLE users
ADD COLUMN age INT;
```

Rename a column:

```
ALTER TABLE users
RENAME COLUMN age TO user_age;
```

Delete a column:

```
ALTER TABLE users
DROP COLUMN user_age;
```

3.8 Best Practices

- Give tables meaningful names.
- Use singular or plural naming consistently.
- Always define a primary key.
- Choose appropriate data types.
- Keep related information together.

Common Mistakes

- Using TEXT everywhere.
- Forgetting PRIMARY KEY.
- Choosing VARCHAR lengths without planning.
- Mixing unrelated data into one table.

Interview Questions

1. What is SERIAL?
2. Difference between VARCHAR and TEXT?
3. When would you use BOOLEAN?
4. Why define a PRIMARY KEY?
5. Difference between INT and NUMERIC?

Practice Exercises

1. Create a database called company.
2. Create an employees table.
3. Add columns for salary and joining_date.
4. Insert three employee records.
5. Rename a column and then remove it.

Chapter Summary

You learned how to create databases and tables, choose PostgreSQL data types, insert records, modify table structures with ALTER TABLE, and follow good schema design practices.

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 4: Constraints & Identity Columns

4.1 What are Constraints?

Constraints are rules applied to table columns to maintain data integrity. They prevent invalid or inconsistent data from being stored.

4.2 PRIMARY KEY

A PRIMARY KEY uniquely identifies each row.

Example:

```
CREATE TABLE users(
id SERIAL PRIMARY KEY,
name VARCHAR(100)
);
```

Rules:
• Unique
• Cannot be NULL
• One primary key per table

4.3 FOREIGN KEY

A FOREIGN KEY creates relationships.

```
CREATE TABLE orders(
id SERIAL PRIMARY KEY,
user_id INT REFERENCES users(id)
);
```

It ensures every order belongs to a valid user.

4.4 UNIQUE

UNIQUE prevents duplicate values.

```
CREATE TABLE users(
email VARCHAR(255) UNIQUE
);
```

Useful for emails, usernames and phone numbers.

4.5 NOT NULL

NOT NULL ensures a value must always be provided.

```
name VARCHAR(100) NOT NULL
```

Without NOT NULL, PostgreSQL allows NULL values.

4.6 DEFAULT

DEFAULT provides a value when none is supplied.

is_active BOOLEAN DEFAULT TRUE

created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

4.7 CHECK

CHECK validates values.

salary INT CHECK(salary >= 0)

age INT CHECK(age >= 18)

Only rows satisfying the condition are accepted.

4.8 GENERATED AS IDENTITY

Modern PostgreSQL recommends:

id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY

instead of SERIAL.

Benefits:
• SQL standard
• Better sequence management
• Preferred for new projects

4.9 SERIAL vs IDENTITY

SERIAL
• Older PostgreSQL feature
• Still supported

IDENTITY
• SQL standard
• Recommended for modern applications

Both automatically generate ids.

4.10 Combining Constraints

Example:

```
CREATE TABLE employees(
id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(255) UNIQUE,
salary INT CHECK(salary>=0),
is_active BOOLEAN DEFAULT TRUE
);
```

Best Practices

✓ Always define a primary key.
✓ Use UNIQUE for login fields.
✓ Use NOT NULL where appropriate.
✓ Validate business rules with CHECK.
✓ Prefer IDENTITY for new PostgreSQL projects.

Common Mistakes

• Forgetting NOT NULL.
• Using SERIAL without understanding IDENTITY.
• Missing FOREIGN KEY relationships.
• Not validating numeric values.

Interview Questions

1. Difference between PRIMARY KEY and UNIQUE?
2. Difference between SERIAL and IDENTITY?
3. Why use CHECK?
4. What does DEFAULT do?
5. Why are FOREIGN KEY constraints important?

Practice Exercises

1. Create a products table using IDENTITY.
2. Add UNIQUE to the email column.
3. Prevent negative prices using CHECK.
4. Add DEFAULT TRUE to an is_active column.
5. Create an orders table with a foreign key to users.

Chapter Summary

You learned PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, DEFAULT, CHECK, GENERATED ALWAYS AS IDENTITY, and the difference between SERIAL and IDENTITY.

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 5: node-postgres (pg)

5.1 What is node-postgres?

node-postgres (pg) is the official PostgreSQL client for Node.js. It allows Node.js applications to connect to PostgreSQL, execute SQL queries and receive results.

5.2 Installation

Install using:

npm install pg

This package is used in Express applications to communicate with PostgreSQL.

5.3 Connecting to PostgreSQL

Example:

```
const { Pool } = require('pg');

const pool = new Pool({
host:'localhost',
port:5432,
user:'postgres',
password:'your_password',
database:'practice'
});
```

5.4 Using Environment Variables

Never hard-code credentials.

Use a .env file:
DATABASE_URL=postgres://user:password@localhost:5432/practice

or individual variables such as DB_HOST, DB_USER, DB_PASSWORD and DB_NAME.

5.5 Running Queries

Example:

```
const result = await pool.query(
'SELECT * FROM users WHERE id=$1',
[id]
);

console.log(result.rows);
```

5.6 Understanding the Result

pool.query() returns an object.
Useful properties:
• rows - actual data
• rowCount - affected/returned rows

• command - SQL command executed

5.7 Async/Await

Database operations take time. Use async/await so JavaScript waits for the query result before continuing.

5.8 Error Handling

Wrap database code in try...catch.
Return meaningful HTTP status codes from controllers when queries fail.

5.9 Best Practices

• Share one Pool instance.
• Store SQL separately when appropriate.
• Use parameterized queries.
• Close clients when using Client.
• Keep credentials in .env.

Common Mistakes

• Creating a new Pool in every request.
• Hard-coding passwords.
• Forgetting await.
• Ignoring errors.
• Accessing result instead of result.rows.

Interview Questions

1. What is node-postgres?
2. Why use async/await with database queries?
3. Why should credentials be stored in .env?
4. What does pool.query() return?
5. Why is result.rows commonly used?

Practice Exercises

1. Install pg.
2. Create a Pool connection.
3. Execute SELECT * FROM users.
4. Print result.rows.
5. Move database credentials into a .env file.

Chapter Summary

You learned what node-postgres is, how to install it, connect to PostgreSQL, execute queries, use result.rows, apply async/await, handle errors and follow backend best practices.

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 6: Pool vs Client

6.1 Why Do We Need a Database Connection?

Before Node.js can execute SQL, it must establish a connection with PostgreSQL. The pg library provides two approaches: Client and Pool.

6.2 What is Client?

Client represents a single database connection.

Typical flow:
1. Create Client
2. Connect
3. Execute query
4. End connection

Useful for scripts, migrations and one-time tasks.

6.3 Client Example

```
const { Client } = require('pg');
const client = new Client(config);
await client.connect();
const result = await client.query('SELECT * FROM users');
await client.end();
```

6.4 What is Pool?

Pool manages multiple reusable database connections. Instead of creating a new connection for every request, connections are reused automatically.

6.5 Pool Example

```
const { Pool } = require('pg');
const pool = new Pool(config);
const result = await pool.query('SELECT * FROM users');
```

6.6 Why Pool is Preferred

In an Express server, hundreds of users may send requests simultaneously. Creating a new connection for every request is slow and wastes resources. A Pool keeps a set of ready-to-use connections, improving performance and scalability.

6.7 Client vs Pool

Client:
• Single connection
• Manual connect/end
• Best for scripts

Pool:
• Multiple reusable connections
• Automatic connection management
• Best for web applications

6.8 Lifecycle

Client:
Connect → Query → End

Pool:
Create Pool Once → Query Many Times → Reuse Connections

6.9 Best Practices

• Create one shared Pool instance.
• Export it from db.js.
• Reuse it in queries/controllers.
• Do not create a Pool inside every request handler.

6.10 Common Mistakes

• Forgetting client.end().
• Creating multiple Pool instances.
• Opening a new connection for every API call.
• Hard-coding credentials instead of using .env.

Interview Questions

1. What is the difference between Pool and Client?
2. Why is Pool preferred in Express applications?
3. When would you use Client instead of Pool?
4. What happens if connections are never closed?
5. Why should there usually be only one Pool instance?

Practice Exercises

1. Create a Client and fetch all users.
2. Convert the code to use Pool.
3. Export a shared Pool from db.js.
4. Explain the Client and Pool lifecycle in your own words.
5. Identify which approach should be used for an Express REST API and why.

Chapter Summary

You learned:
• Client vs Pool
• Connection lifecycle
• Why Pool is recommended for web servers
• Performance benefits
• Common mistakes
• Interview-focused concepts

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 7: Parameterized Queries & SQL Injection

7.1 Why Security Matters

Applications often receive user input from forms, URLs and APIs. Treat every input as untrusted and never place it directly into an SQL query.

7.2 What is SQL Injection?

SQL Injection is an attack where malicious SQL is inserted into application input to change the intended query. This can expose, modify or delete data.

7.3 Unsafe Example

 Bad:
```
const query =
`SELECT * FROM users WHERE email='${email}'`;
```

If email contains malicious SQL, the query can behave unexpectedly.

7.4 Parameterized Queries

 Safe:
```
const result = await pool.query(
'SELECT * FROM users WHERE email = $1',
[email]
);
```

$1 is a placeholder. PostgreSQL safely treats the value as data, not executable SQL.

7.5 Multiple Parameters

Example:
```
await pool.query(
'SELECT * FROM users WHERE email=$1 AND age>$2',
[email, age]
);
```

$1, $2, $3 ... correspond to array positions.

7.6 How pool.query() Works

pool.query(text, values)

text: SQL statement with placeholders.
values: Array of parameters.
The pg library sends them separately to PostgreSQL, preventing SQL injection.

7.7 Real-world Examples

Login:
```
SELECT * FROM users WHERE email=$1;
```

Search:
```
SELECT * FROM users WHERE name ILIKE $1;
```

Update:
UPDATE users SET name=$1 WHERE id=$2;

7.8 Best Practices

• Always use placeholders.
• Validate input types.
• Never trust client input.
• Keep secrets in .env.
• Return generic error messages to clients.

7.9 Common Mistakes

• Building SQL with string concatenation.
• Forgetting parameter arrays.
• Exposing database errors directly.
• Assuming frontend validation is enough.

Interview Questions

1. What is SQL Injection?
2. Why are parameterized queries safer?
3. What does $1 represent?
4. Explain pool.query(text, values).
5. Is frontend validation enough to stop SQL Injection? Why?

Practice Exercises

1. Convert a string-concatenated SELECT query into a parameterized query.
2. Write a safe UPDATE query using placeholders.
3. Write a DELETE query using $1.
4. Explain why placeholders prevent SQL Injection.
5. Identify unsafe SQL in sample code.

Chapter Summary

You learned:
• SQL Injection basics
• Safe vs unsafe queries
• $1, $2 placeholders
• pool.query(text, values)
• Security best practices
• Common interview topics

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 8: Seed Scripts

8.1 What is a Seed Script?

A seed script populates a database with initial tables and sample data. It allows every developer on a team to start with the same development data.

8.2 Why Use Seed Scripts?

• Faster project setup
• Consistent test data
• Easy database reset during development
• Helpful for demos and automated testing

8.3 Typical Workflow

1. Create the database.
2. Run the seed script.
3. Tables are created if they do not already exist.
4. Sample records are inserted.
5. Start the application.

8.4 CREATE TABLE IF NOT EXISTS

Using IF NOT EXISTS prevents PostgreSQL from trying to create a table that already exists.

Example:
```
CREATE TABLE IF NOT EXISTS users(
id SERIAL PRIMARY KEY,
name VARCHAR(100),
email VARCHAR(255)
);
```

8.5 Inserting Multiple Records

Example:
```
INSERT INTO users(name,email)
VALUES
('Nikhil','nikhil@gmail.com'),
('Rahul','rahul@gmail.com'),
('Aman','aman@gmail.com');
```

8.6 seed.js Structure

Typical flow:
• Import Pool/Client
• Connect
• Execute SQL
• Handle errors
• Close connection (if using Client)
• Exit process

8.7 Resetting Development Data

During development you may delete existing data and rerun the seed script. Be careful not to run destructive seed scripts against production databases.

8.8 Best Practices

• Keep seed scripts idempotent where possible.
• Separate schema creation and sample data.
• Store SQL in readable blocks.
• Use transactions for large seed operations.

Common Mistakes

• Forgetting IF NOT EXISTS.
• Inserting duplicate sample data repeatedly.
• Running development seed scripts in production.
• Forgetting to close Client connections.

Interview Questions

1. What is a seed script?
2. Why use CREATE TABLE IF NOT EXISTS?
3. Why insert sample data?
4. Should seed scripts run in production?
5. When would you use a transaction in a seed script?

Practice Exercises

1. Write a seed script to create a users table.
2. Insert three users.
3. Add an employees table.
4. Make the script safe to run twice.
5. Explain why IF NOT EXISTS is useful.

Chapter Summary

You learned the purpose of seed scripts, how they create tables and insert sample data, why IF NOT EXISTS is important, and how to organize and safely run seed scripts.

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 9: Express + PostgreSQL Architecture

9.1 Why Use Architecture?

As applications grow, putting all code in one file becomes difficult to maintain. Separating responsibilities into layers makes code easier to understand, test and extend.

9.2 Typical Folder Structure

```
project/
├── db/
│   └── db.js
├── controllers/
├── routes/
├── queries/
├── app.js
└── server.js
```

9.3 Request Flow

```
Client
↓
Express Route
↓
Controller
↓
queries.js
↓
db.js (Pool)
↓
PostgreSQL
↓
Controller
↓
JSON Response
```

9.4 Routes

Routes define the API endpoints such as GET /users or POST /users. Their responsibility is to map incoming requests to the correct controller function.

9.5 Controllers

Controllers contain the application logic. They validate input, call database functions, handle errors and send HTTP responses.

9.6 queries.js

Store SQL queries in a separate file. This keeps controllers clean and allows SQL logic to be reused in multiple controllers.

9.7 db.js

Create one shared Pool instance in db.js and export it. Every query file imports the same Pool instead of creating new connections.

9.8 Example Flow

Frontend sends GET /users
→ Route matches /users
→ Controller calls getAllUsers()
→ queries.js executes:
SELECT * FROM users;
→ PostgreSQL returns rows
→ Controller sends:
res.status(200).json(rows)

9.9 Error Handling

Wrap controller logic in try...catch. Return appropriate status codes such as 200, 201, 400, 404 and 500. Never expose sensitive database details in responses.

9.10 Best Practices

• Thin controllers.
• Reusable query functions.
• One Pool instance.
• Environment variables for credentials.
• Clear separation of concerns.

Common Mistakes

• Writing SQL directly in routes.
• Creating a Pool inside controllers.
• Mixing validation, SQL and response logic together.
• Returning raw database errors.

Interview Questions

1. Why separate routes, controllers and queries?
2. Why create only one Pool instance?
3. What is the responsibility of a controller?
4. Why shouldn't SQL be written inside routes?
5. Describe the request flow from client to database.

Practice Exercises

1. Create routes for GET and POST users.
2. Move SQL into queries.js.
3. Create db.js exporting one Pool.
4. Trace a request from frontend to PostgreSQL.
5. Refactor a single-file Express app into this architecture.

Chapter Summary

You learned a scalable Express + PostgreSQL architecture, the role of each layer, request flow, separation of concerns, and backend best practices.

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 10: Building CRUD APIs with pg

10.1 Goal

Build REST APIs using Express, node-postgres and PostgreSQL following the architecture: Routes → Controllers → Queries → db.js → PostgreSQL.

10.2 Project Structure

routes/
controllers/
queries/
db/
app.js
server.js

10.3 GET /users

Flow:
Client → Route → Controller → getAllUsers() → pool.query('SELECT * FROM users') → res.status(200).json(rows)

10.4 POST /users

Read req.body, validate input, execute INSERT query using parameterized placeholders, return status 201 with the created user.

10.5 PUT /users/:id

Read req.params.id and req.body, update the row with WHERE id=$1, return the updated record.

10.6 DELETE /users/:id

Delete using:
DELETE FROM users WHERE id=$1;
Return success status and message or deleted record.

10.7 HTTP Status Codes

200 OK
201 Created
204 No Content
400 Bad Request
404 Not Found
500 Internal Server Error

10.8 Error Handling

Wrap controllers in try...catch. Validate inputs before querying. Return consistent JSON error responses.

10.9 Validation

Check required fields, convert id from req.params to Number where appropriate, and reject invalid requests early.

10.10 Best Practices

• Keep routes thin.
• Controllers handle business logic.
• SQL stays in queries.js.
• Use one Pool instance.
• Always use parameterized queries.

Interview Questions

1. Explain the flow of a GET request.
2. Why use parameterized INSERT queries?
3. Which status code is returned after creating a resource?
4. Where should SQL queries be written?
5. Why is validation important?

Practice Exercises

1. Create GET /users.
2. Create POST /users.
3. Create PUT /users/:id.
4. Create DELETE /users/:id.
5. Add proper error handling and status codes.

Chapter Summary

You learned how to structure CRUD APIs using Express and node-postgres, organize code into layers, validate input, use parameterized queries, and return appropriate HTTP status codes.

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 11: Search, Pagination & API Design

11.1 Why Search APIs?

Applications rarely return every record. Users search by name, email, category or other fields. Search APIs improve usability and reduce unnecessary data transfer.

11.2 Query Parameters

Search and pagination are commonly passed as query parameters.
Example:
GET /users?search=nik&page;=2&limit;=10
Access them with req.query.search, req.query.page and req.query.limit.

11.3 Building a Search Query

Use parameterized queries.
Example:
SELECT * FROM users WHERE name ILIKE $1;
Value: ['%nik%']
ILIKE performs case-insensitive matching in PostgreSQL.

11.4 Pagination

Use LIMIT and OFFSET.
OFFSET=(page-1)*limit
Example:
SELECT * FROM users LIMIT $1 OFFSET $2;

11.5 Combining Search and Pagination

Search first, then paginate.
Example:
SELECT * FROM users WHERE name ILIKE $1 LIMIT $2 OFFSET $3;

11.6 API Response Design

Return consistent JSON.
{
success:true,
data:[...],
page:2,
limit:10
}
Consistency makes frontend development easier.

11.7 Validation

Validate page and limit as positive integers. Provide sensible defaults when parameters are missing.

11.8 Performance Tips

Index searchable columns, avoid returning unnecessary fields, paginate large datasets, and avoid expensive wildcard searches where possible.

11.9 Best Practices

Keep filtering logic in controllers or query helpers, always use parameterized queries, document query parameters, and return meaningful HTTP status codes.

Common Mistakes

Concatenating search strings into SQL, forgetting LIMIT, negative page numbers, inconsistent response formats, and exposing database errors.

Interview Questions

1. Why use query parameters?
2. Difference between req.params and req.query?
3. How do LIMIT and OFFSET work?
4. Why use ILIKE instead of LIKE?
5. How would you design a paginated API response?

Practice Exercises

1. Create GET /users?search=rahul.
2. Add page and limit parameters.
3. Return only 10 users per page.
4. Add sorting by name.
5. Design a consistent JSON response.

Chapter Summary

You learned how to build searchable and paginated APIs using query parameters, parameterized SQL, LIMIT/OFFSET, consistent response formats and backend best practices.

The Complete SQL, PostgreSQL & Prisma ORM Handbook

Volume 2: PostgreSQL & node-postgres
Chapter 12: PostgreSQL Interview Guide & Mini Project

12.1 PostgreSQL Revision Sheet

Core Topics:
• PostgreSQL architecture
• pgAdmin & psql
• Databases & Tables
• Data Types
• Constraints
• Pool vs Client
• Parameterized Queries
• Seed Scripts
• Express Architecture
• CRUD APIs
• Search & Pagination

12.2 Common Interview Questions

1. What is PostgreSQL?

2. Difference between SQL and PostgreSQL?

3. Why use Pool instead of Client?

4. What is SQL Injection?

5. How do parameterized queries prevent SQL Injection?

6. Difference between req.params and req.query?

7. Why use queries.js?

8. Why create only one Pool instance?

9. When do you use CREATE TABLE IF NOT EXISTS?

10. Explain the request flow in an Express + PostgreSQL app.

12.3 Mini Project - User Management API

Build an API with:
• GET /users
• GET /users?search=nik&page;=1&limit;=10
• POST /users
• PUT /users/:id
• DELETE /users/:id

Folder Structure:
routes/
controllers/
queries/
db/
app.js
server.js

Use:
• Pool

• Parameterized queries
• try...catch
• HTTP status codes
• Validation

12.4 Best Practices

✓ One shared Pool
✓ Credentials in .env
✓ Parameterized queries only
✓ Thin controllers
✓ Reusable SQL functions
✓ Consistent JSON responses
✓ Validate input
✓ Return correct status codes

12.5 Common Mistakes

• Creating a Pool per request
• Forgetting await
• String concatenation in SQL
• Missing error handling
• Returning raw database errors
• Forgetting to convert numeric route parameters when needed

12.6 Self Assessment

You should now be able to:
• Connect Node.js to PostgreSQL
• Build CRUD APIs
• Organize an Express backend
• Write safe SQL queries
• Build search and pagination
• Explain Pool vs Client confidently

Volume 2 Completed!

Congratulations!

You have completed Volume 2.

Next Volume:
Prisma ORM

Topics:
• Installation
• Schema
• Models
• Relations
• CRUD
• Filtering
• Pagination
• Prisma Studio
• Raw SQL
• Transactions
• Express + Prisma